Experimental Fault Injection in an FPGA Based Implementation of Tank Battalion

Justin Spenker
University of California Davis
Computer Engineering
Davis, CA
Jdspenker@ucdavis.edu

Kyle Osborn
University of California Davis
Computer Engineering
Davis, CA
kyle.cameron.osborn@gmail.com

***Abstract*— Field programmable gate arrays (FPGAs) are widely used in embedded and specialized hardware systems, where functional failures can arise from logic design errors, timing violations, or physical faults in configuration or interconnection resources. Diagnosing these failures is challenging because many hardware faults manifest as symptoms that resemble software defects. This work investigates fault behavior in FPGA based systems through controlled fault injection using an implementation of the classic arcade game Tank Battalion. The system reproduces the original hardware architecture, including CPU control logic, video timing circuits, graphics generation, and memory mapped I/O. Using this platform, several fault models including stuck-at faults and transient bit flips are introduced at key points in the design such as synchronization signals, blanking logic, and internal counter stages. The resulting behavior is analyzed through observable system outputs including display stability, graphical corruption, and gameplay anomalies. Experimental results demonstrate that faults in timing and rendering subsystems produce distinct and diagnosable visual artifacts. These observations illustrate how controlled fault injection combined with architectural analysis can provide a practical framework for debugging and validating FPGA-based hardware systems.**

I. Introduction

Field programmable gate arrays (FPGAs) are widely used in embedded and specialized hardware systems due to their flexibility and ability to implement complex digital architecture. However, debugging failures in FPGA based systems can be challenging because faults can appear to be software related errors may actually originate from a physical issue within the board. Errors such as timing violations, routing delays, memory configuration/corruption, or transient bit flips can produce symptoms which include but not limited to unstable outputs, intermittent glitches, or incorrect system behavior.

Understanding how such faults manifest is an important step in the development of reliable FPGA systems and effective diagnostic workflows. One useful approach which is used several times throughout this experiment is controlled fault injection, where specific faults are intentionally introduced into the system resulting in observable behavior. This allows engineers and researchers to study how different fault mechanisms propagate through hardware systems and how they appear at the systems output.

In this work, fault behavior is investigated using FPGA implementation of the arcade game Tank Battalion. The system replicates the physical hardware architecture including CPU control logic, video timing generation, memory mapping, I/O, and graphics rendering circuits. This paper demonstrates how controlled fault injections using a virtual example can be used to analyze and diagnose faults in FPGA based hardware systems. Several fault models are used at key points within the video timing and rendering logic to generate an observable behavior. The resulting system behavior is then analyzed through display instabilities, graphical corruption, and gameplay anomalies. These experiments illustrate how architectural knowledge combined with targeted testing can aid in identifying the root cause of faults in complex digital systems.

II. System Architecture

The architecture of the original Tank Battalion hardware was first analyzed using publicly available board schematics [1] which is composed of several discrete subsystems including 6502 based CPU, ROM and RAM, video timing counters, sprite and tile rendering logic, and analog audio circuitry. Unlike modern graphics systems that rely on framebuffers, the video signals generated directly from hardware timing signals derived from horizontal and vertical counters. These counters produce synchronization and blanking signals that determine when pixels are visible to the display and clock timing for the internal components.

Using the published board schematics[1], the system was divided into functional blocks and reproduced in an FPGA environment. In this project the original hardware was reproduced using a DE10-Nano platform within the MiSTer environment. The open source Tank Battalion HDL core recreates the behavior of the original board by implementing the CPU subsystem, video timing logic, memory, sprite rendering logic in synthesizable Verilog/SystemVerilog modules. This FPGA implementation allows controlled manipulation of internal signals, making it possible to inject artificial faults and observe the resulting system behavior.

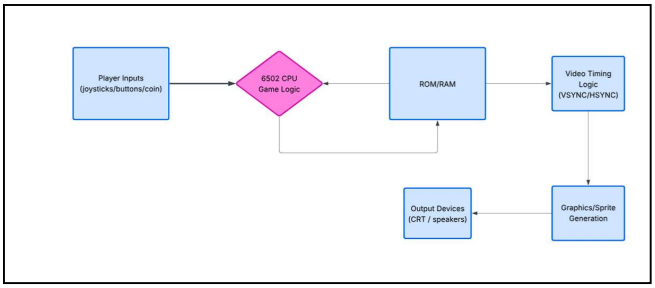


Figure 1. Data flow and Pipeline

Figure 1 illustrates the simplified data flow of the system. Player inputs will first be processed by the 6502 CPU, which executes game logic and updates system memory. The video timing generator produces horizontal and vertical synchronization signals that determine the current pixel location on the display. Graphic logic then reads tiles and sprite data from ROM and RAM based on these timing signals. The resulting timing signals are converted to pixel values and combined with synchronization signals produce the final video output sent to the CRT display.

The video subsystem relies on a chain of counters and decoding logic that synchronizes blanking signals required for CRT displays. Horizontal counters determine the position within each scan line and generate HSYNC and HBLANK signals. Vertical counters track current frame position and use to generate similar signals VSYNC and VBLANK. These signals control when pixel data is visible and when the display beam is retraced to the beginning of the new line. Because these timing signals directly control the monitor synchronization process, faults in this subsystem often produce noticeable system failures.

Sound generation in the Tank Battalion platform is controlled through memory mapped output latches. The CPU writes values to specific addresses, which set digital control lines such as SHOOT, HIT, or engine sounds. These control lines enable oscillators which envelope shaping audio signals. Because the sound system is controlled through discrete latch signals, faults in these control lines can produce observable effects such as continuous sound playback or missing audio events.

Understanding the interaction between these subsystems is critical for diagnostic faults in FPGA based systems. Because synchronization signals, memory access paths, and control logic directly influence the external behavior of the system, faults within these regions can produce distinct and measurable outputs. To investigate these effects, controlled fault injection was preformed within the FPGA implementation by intentionally introducing faults at specific points. This makes it possible to observe how these errors propagate through the system and how they manifest in visible display artifacts or gameplay anomalies.

III. Fault Injection

Fault injection is a widely used technique for evaluating the reliability and behavior of digital systems under abnormal operating conditions. Rather than waiting for faults to occur naturally, faults are intentionally introduced into a system in a controlled manner to observe how errors propagate and how they affect system outputs. In this work, fault injection was applied to the FPGA implementation of the Tank Battalion system in order to study how faults in key subsystems, particularly video timing and graphics generation manifest as observable display and gameplay anomalies.

Using the FPGA implementation running within the MiSTer framework based off a software simulation of Tank Battalion, several synchronization related fault conditions were introduced in order to observe video system behavior. Specifically stuck at faults targeting HSYNC, HBLANK, VSYNC signals within the video timing logic. Based on theoretical operation of CRT video timing systems, forcing these signals to incorrect logic should render the display unstable causing horizontal or vertical synchronization, screen collapsing, continuous blanking, or visible scan artifacts such as ghosting or tearing. However, during experimentation the system remained fully operational and no significant graphical corruption or synchronization failure were observed.

Further investigation revealed that the MiSTer FPGA framework includes protective handling or synchronization signals to prevent invalid timing conditions from reaching the physical display interface. These safeguards are implemented to protect connected CRT monitors from potentially damaging signal conditions. As a result, the injected HSYNC, HBLANK, and VSYNC faults were effectively masked by the output pipeline, preventing the expected catastrophic display failures from occurring.

In contrast, manipulation of the VBLANK signal produced a visible but not catastrophic graphical anomaly. When VBLANK was forced the game remained playable but slight red anomalies appeared along the lower borders of textures. This behavior suggests that within the implementation video pipeline the VBLANK signal primarily controls frame boundary conditions rather than globally gating pixel output. The observed artifact is consistent with incorrect vertical blank boundary handling, resulting in pixel data leakage near the end of the frames.

Another injection point evaluated during experimentation involved stuck at faults on player movement inputs. These signals correspond to the direction of movement provided by the joystick, which allows the player to move the tank in four different directions. By focusing these input signals to constant logic values within the FPGA implementation, the system behaves as if the corresponding button is continuously pressed or inactive. These faults are comparatively easy to diagnose because the resulting behavior maps directly to the affected input path and does not significantly alter video timing or rendering behavior.

Fault Location	Fault Type	Expected Effect	Observed Effect
HSYNC	Stuck at	Loss of horizontal lock	No visible effect due to MiSTer protection
VSYNC	Stuck at	Vertical roll / collapse	No visible effect due to MiSTer protection
HBLANK	Stuck at	Black Screen or retrace artifacts	No visible effect due to MiSTer protection
VBLANK	Stuck at	Frame blanking failure	Red artifacts at lower boarder textures
Movement input	Stuck at	Continuous or absent movement	As expected

IV. Conclusion

This work demonstrated how controlled fault injection can be used to study the behavior of FPGA based hardware systems through a practical implementation of the retro game Tank Battalion. By recreating the original hardware architecture within the MiSTer FPGA framework, it became possible to introduce faults at targeted locations and observe how these faults propagated through to produce visible behavior effects.

Several fault models were explored, including being stuck at faults in synchronization and input lines while theoretical analysis predicted that certain faults, such as HYSNC or VSYNC failures, would render the display unusable, while experimental observations showed that the MiSTer framework incorporates protective mechanisms that prevent invalid synchronization signals. As a result, these faults were masked during the experimentation and did not yield expected results. Other faults, such as manipulation of the VBLANK signal, produced observable artifacts in the video output, demonstrating how subtle timing changes can affect rendering behavior.

References

[1] Nic24-rgb, "Tank Battalion Board Schematics," GitHub repository. [On Available: https://github.com/nic24rgb/TankBatt/blob/main/TB%20board.pdf. Accessed: Feb. 2026.

[2] Nic24-rgb, "Tank Battalion Board Schematics," GitHub repository. [On Available: https://github.com/nic24-rgb/Arcade-TankBattalion-MiSTer. Accessed: Feb. 2026.

[3] J. Qin, "A Brief Introduction to Application-Dependent FPGA Testing," Dept. of Electrical and Computer Engineering, Auburn University, Auburn. [Online]. Available: https://www.eng.auburn.edu/~agrawvd/COURSE/E7250_06/TERM/Qin_ADFPGAT.pdf.
Accessed: Feb. 2026.K. Elissa, "A Brief Introduction to Application-Dependent FPGA Testing".